

A BEGINNER'S FIELD GUIDE

Machine Learning, From Zero

Plain-English foundations for people who have never written a line of ML.

Everything you need is in these pages. No prior maths, no jargon dumped on you, and a small project you can run yourself by the end.

Phase 1 • ML Foundations

Before you start

This is Phase 1 of a four-part guide. It assumes you know nothing about machine learning. By the last page you will understand the core ideas, recognise the main algorithms, know how to tell a good model from a bad one, and you will have trained your own model on your computer.

Who this is for

Anyone new to the field: a curious developer, an analyst, a manager who wants to actually understand what their data team is talking about, or a student. If a term ever feels heavy, there is a plain-English glossary at the back.

How the four phases fit together

Phase	What it gives you
1 • Foundations (you are here)	The mental model: what ML is, the main algorithms, and how success is measured.
2 • Python for ML	The practical toolkit: pandas, NumPy, plotting, and scikit-learn workflows.
3 • Applied ML	Real use cases: fraud/anomaly detection, forecasting, classification, text/NLP, and running models in production.
4 • ML for Business	Strategy: ROI, roadmaps, talking to stakeholders, ethics, and the platform landscape.

A NOTE ON SOURCES

There is no single "official website" for machine learning — it is a field, not a product. The most widely used and trusted reference for the classic algorithms and metrics in this phase is the official documentation of **scikit-learn** (scikit-learn.org), the standard Python ML library. The current stable release is **version 1.8** (December 2025), supporting Python 3.11–3.14. All definitions here follow that documentation.

WHAT YOU'LL BE ABLE TO DO AFTER PHASE 1

Explain ML to a colleague without hand-waving · name the right algorithm for a problem · read a model's score and say whether it is actually good · train and test a real classifier in about 20 lines of Python.

What's inside

A **Core Concepts** — the mental model

AI vs ML vs Deep Learning · the three styles of learning · training vs inference · overfitting & underfitting

B **Key Algorithms** — intuition only, no maths

Linear regression · logistic regression · decision trees · K-Means clustering · recommendation systems

C **Evaluation Metrics** — how to judge a model

Confusion matrix · accuracy · precision & recall · F1 · AUC-ROC · RMSE & MAE

D **Hands-On Project** — "The Flower Sorter"

Train, test and evaluate your first model · see overfitting happen live · stretch challenge

+ **Plain-English Glossary** & what comes next

Core Concepts

The four ideas that everything else in machine learning rests on.

A.1 AI vs Machine Learning vs Deep Learning

These three words get used as if they mean the same thing. They don't. They sit inside one another like nesting dolls.

Artificial Intelligence (AI) — the big outer doll

AI is the broad goal of getting computers to do things that normally need human intelligence: understanding language, recognising a face, making a decision. AI is the *ambition*. It includes everything from simple rule-based systems ("if temperature > 30, turn on the fan") to the most advanced models.

Machine Learning (ML) — a doll inside AI

ML is one particular way of achieving AI. Instead of a human writing every rule by hand, you **show the computer many examples and let it work out the rules itself**. That is the entire trick. You don't tell it "a cat has whiskers and pointy ears"; you show it thousands of labelled cat photos and it figures out the pattern.

Deep Learning — a smaller doll inside ML

Deep learning is one family of ML methods built on *neural networks* — layers of simple calculations loosely inspired by the brain. It shines on messy, high-volume data like images, audio and text, but it usually needs lots of data and computing power. It is powerful but not always the right tool.

PLAIN-ENGLISH ANALOGY

Think of **transport** as AI (the goal: get from A to B). **Motor vehicles** are machine learning (one popular way to do it). **Electric cars** are deep learning (a specific, modern, powerful kind of motor vehicle). Every electric car is a motor vehicle, and every motor vehicle is transport — but not all transport is an electric car.

KEY IDEA

AI $\supset$ ML $\supset$ Deep Learning. Traditional programming is *rules in* $\rightarrow$ *answers out*. Machine learning flips it: *answers (examples) in* $\rightarrow$ *rules out*.

A.2 The three ways a machine can learn

Almost every ML problem falls into one of three styles. Knowing which one you're in tells you which tools to reach for.

1. Supervised learning — learning with an answer key

You give the model examples that already include the correct answer (the "label"). It learns the relationship between the inputs and the answer, so it can predict the answer for new, unseen inputs. This is by far the most common style in practice.

- **Predicting a number** (called *regression*): house price, tomorrow's sales, a patient's blood sugar.
- **Predicting a category** (called *classification*): spam or not spam, which of three flower species, will this customer leave or stay.

2. Unsupervised learning — finding structure with no answer key

Here there are no labels. You hand the model a pile of data and ask, "what natural groups or patterns are in here?" It is discovery, not prediction. The classic job is *clustering* — for example, grouping customers into segments you didn't know existed.

3. Reinforcement learning — learning by trial, reward and error

An "agent" tries actions in an environment and gets rewards or penalties. Over many attempts it learns a strategy that earns the most reward. This is how programs learn to play games or how robots learn to walk. It is powerful but more specialised, and you'll meet it less often in everyday business work.

PLAIN-ENGLISH ANALOGY

Supervised = studying with flashcards that have the answer on the back. **Unsupervised** = being handed a messy drawer of photos and sorting them into piles that "feel" similar. **Reinforcement** = learning to ride a bike: no instruction manual, just wobble, fall, adjust, and keep what worked.

Style	Do you have the answers?	Typical question	Example
Supervised	Yes (labelled)	"What is the answer for this new case?"	Is this email spam?
Unsupervised	No	"What groups exist in this data?"	Which customers are similar?
Reinforcement	No — just rewards	"Which actions earn the most reward?"	Learning to play chess

A.3 Training vs inference

A model has two stages in its life, and beginners often blur them. Keeping them separate will save you a lot of confusion later.

Training — the learning phase

Training is when the model studies your example data and adjusts itself to capture the patterns. It can be slow and is usually done once (or occasionally repeated). The output of training is a saved *model* — basically a file full of learned settings.

Inference — the using phase

Inference (also called *prediction*) is when you feed the trained model a brand-new input and it gives you an answer. It is fast, and it happens every time someone uses your app.

PLAIN-ENGLISH ANALOGY

Training is **studying for the exam** — long, effortful, done in advance. Inference is **answering a single question on exam day** — quick, using what you already learned. You study once and answer many times.

A.4 Overfitting and underfitting

This is the single most important idea for telling a good model from a bad one. The goal of ML is not to do well on data you've already seen — it is to do well on data you *haven't* seen yet. That ability is called **generalisation**.

Underfitting — too simple

The model is too basic to capture the real pattern. It does poorly on both the data it studied and on new data. It's like revising for a history exam by memorising only the single sentence "stuff happened in the past." Too vague to be useful.

Overfitting — too clever for its own good

The model memorises the training examples in detail — including their random quirks and noise — instead of learning the general rule. It looks *brilliant* on the data it studied but falls apart on anything new. It's like memorising the exact answers to last year's exam paper word-for-word, then panicking when this year's questions are phrased differently.

KEY IDEA

The sweet spot is in the middle: complex enough to capture the real pattern, simple enough to ignore the noise. We *detect* overfitting by always testing the model on data it never saw during training (more on this in the project).

Here is overfitting happening for real. We trained the same kind of model on the same data, only changing how complex we let it become (its "depth"). Watch the two columns drift apart:

REAL OUTPUT — SAME MODEL, INCREASING COMPLEXITY

complexity	score on data it studied	score on NEW unseen data
very low	0.70	0.60 <- underfitting
low	0.96	0.91
just right	0.98	0.98 <- the sweet spot
high	1.00	0.98
very high	1.00	0.98 <- memorising, no gain

Notice how, once the model gets very complex, its score on studied data hits a perfect **1.00** — but its score on new data stops improving. That gap is the fingerprint of overfitting: **great on the practice paper, no better on the real exam**.

PART B

Key Algorithms

Five workhorses of machine learning — explained by intuition and analogy only. No equations.

You do not need to build these from scratch; libraries do that for you. You *do* need to know what each one is for, so you can pick the right one. Here are the five you'll meet most.

B.1 Linear regression — drawing the best straight line

What it's for: predicting a *number* from one or more inputs.

Imagine plotting house size against house price as dots on a chart. Bigger houses tend to cost more, so the dots trend upward. Linear regression simply finds the **single straight line that sits as close as possible to all the dots**. Once you have that line, you can read off a price for any house size.

PLAIN-ENGLISH ANALOGY

It's the line you'd instinctively draw through a scatter of points with a ruler — "roughly, as this goes up, that goes up too." The algorithm just finds the most balanced ruler position mathematically.

Use it when: you want a number and you suspect a roughly straight-line relationship. **Watch out:** if the real relationship is curvy, a straight line will underfit.

B.2 Logistic regression — a line that decides yes or no

What it's for: predicting a *category*, most often a yes/no (will this customer churn? is this email spam?).

Despite the confusing name, this is a *classification* tool, not a number-predictor. Instead of a price, it outputs a **probability between 0 and 1** — for example "0.85 chance this is spam." You then set a cut-off (say 0.5): above it, call it spam; below it, don't.

PLAIN-ENGLISH ANALOGY

Think of a dimmer switch rather than an on/off switch. It doesn't just say "spam"; it says "I'm 85% sure," and you decide how confident you need to be before acting.

Use it when: you need a simple, fast, explainable yes/no model. It's often the first thing practitioners try for classification.

B.3 Decision trees — a flowchart of yes/no questions

What it's for: both categories and numbers; loved for being easy to explain.

A decision tree learns a flowchart of simple questions. "Is the petal longer than 2.5cm? If yes, go left; if no, go right. Is the petal wider than 1.8cm?" ... and so on, until it reaches an answer at the bottom. You can literally read the logic and follow it by hand.

PLAIN-ENGLISH ANALOGY

It's the "20 Questions" game. Each question narrows things down until only one answer remains. That transparency is its superpower — you can show the exact path to a sceptical stakeholder.

WATCH OUT

A tree left to grow unlimited will ask so many oddly specific questions that it memorises the training data — the overfitting we saw earlier. We tame it by limiting its depth (how many questions deep it can go). This is exactly the knob we turned in the A.4 demo.

B.4 K-Means clustering — sorting into groups with no labels

What it's for: unsupervised grouping. You don't have labels; you want the machine to find natural clusters.

You tell it how many groups you want (that's the "K" — say 3). It then drops 3 markers into the data, assigns each data point to its nearest marker, slides each marker to the centre of the points it caught, and repeats until the groups settle. The result: every point belongs to one of K tidy groups.

PLAIN-ENGLISH ANALOGY

Imagine kids scattered across a playground and you place 3 teachers. Each child walks to the nearest teacher; each teacher then shuffles to the middle of their cluster of kids; repeat until nobody needs to move. You've formed 3 groups without ever labelling anyone.

Use it when: customer segmentation, grouping similar documents, or any "what natural types exist here?" question. **Watch out:** you have to choose K yourself, and a poor choice gives meaningless groups.

B.5 Recommendation systems — "people like you also liked..."

What it's for: suggesting items a user is likely to want — the engines behind Netflix, Spotify and online shops.

There are two common intuitions:

- **"Users like you" (collaborative filtering):** find people whose tastes overlap with yours, then recommend things they liked that you haven't tried yet.
- **"Items like this" (content-based):** if you liked a film, recommend other films with similar features (same genre, director, mood).

PLAIN-ENGLISH ANALOGY

Collaborative filtering is a friend saying "we have the same taste in music, you'll love this band." Content-based is a shop assistant saying "you bought this jacket, here's a matching one." Most real systems blend both.

PART C

Evaluation Metrics

A model always produces *some* answer. These tools tell you whether to trust it.

Two families of metrics: one for when the model predicts **categories** (classification), one for when it predicts **numbers** (regression). We'll start with the small table that powers most of the classification metrics.

C.1 The confusion matrix — the foundation

For a yes/no prediction there are exactly four possible outcomes. Laying them in a 2×2 grid — the **confusion matrix** — is the foundation for almost every other classification metric. Imagine a medical test for a disease:

	Model says "YES"	Model says "NO"
Truly YES	True Positive sick & correctly caught ✓	False Negative sick but missed ✗
Truly NO	False Positive healthy, false alarm ✗	True Negative healthy & correctly cleared ✓

KEY IDEA

The two mistakes are not equal. A **false negative** (telling a sick patient they're fine) and a **false positive** (scaring a healthy patient) cause different harm. Good metrics let you care about the right mistake for your problem.

C.2 Accuracy — and why it can lie

Accuracy = the fraction of all predictions that were correct. Simple and intuitive: 90 right out of 100 = 90% accuracy.

WATCH OUT — THE ACCURACY TRAP

Suppose 99% of transactions are legitimate and only 1% are fraud. A lazy model that screams "legitimate!" every single time is **99% accurate** — and utterly useless, because it never catches a single fraud. When your categories are imbalanced, accuracy alone is misleading. That's why the next metrics exist.

C.3 Precision and recall — two honest questions

These two pick apart what accuracy hides. Keep the disease test in mind.

Precision — "when the model says yes, how often is it right?"

Of all the cases the model flagged as positive, what fraction truly were? High precision means few false alarms. Per the scikit-learn documentation, precision is the true positives divided by all the cases the model labelled positive.

Recall — "of all the real yes cases, how many did the model catch?"

Of everything that was truly positive, what fraction did the model find? High recall means few misses. It is the true positives divided by all the cases that were actually positive.

PLAIN-ENGLISH ANALOGY — FISHING WITH A NET

Precision asks: of the fish in my net, how many are the kind I wanted (vs. boots and seaweed)? **Recall** asks: of all the target fish in the whole lake, how many did I actually catch? A tiny careful net has high precision but low recall; dragging the whole lake has high recall but low precision.

You should care most about...	When...	Because a miss/false-alarm means...
Recall (don't miss any)	Detecting cancer, fraud, safety faults	Missing a real case is dangerous; a false alarm is just an extra check.
Precision (don't cry wolf)	Spam filters, recommending content	A false alarm annoys users (real email in spam); a rare miss is tolerable.

C.4 F1 score — one number when you want both

Often you care about precision *and* recall at once. The **F1 score** blends them into a single number. The official scikit-learn documentation describes it as the *harmonic mean* of precision and recall, scored from 0 (worst) to 1 (best).

Why a harmonic mean rather than a plain average? Because it punishes imbalance. If precision is great but recall is terrible, a plain average looks okay, but F1 stays low — it refuses to let one good number hide one bad one. You only get a high F1 when *both* are high.

KEY IDEA

Use **accuracy** for a quick, balanced-data gut check. Use **precision/recall** when one type of mistake matters more. Use **F1** when you need a single fair score that respects both.

C.5 AUC-ROC — performance across every cut-off

Remember logistic regression outputs a probability, and you choose a cut-off (0.5, 0.7...) to turn it into yes/no. Different cut-offs trade precision against recall. **AUC-ROC** measures how well the model separates the two classes across *all* possible cut-offs at once, so you don't have to commit to one to judge it.

It's reported as a single number from 0.5 to 1.0:

AUC value	Interpretation
1.0	Perfect separation — flawless ranking.

0.8 – 0.9	Strong, genuinely useful model.
0.5	No better than a coin flip — worthless.

PLAIN-ENGLISH ANALOGY

Pick one real positive and one real negative case at random. AUC is the chance the model gives the positive one a higher score than the negative one. A perfect model always ranks them correctly (1.0); a clueless one is right half the time (0.5).

C.6 RMSE and MAE — for models that predict numbers

When the model predicts a *number* (regression), "right or wrong" doesn't apply — instead we measure **how far off** the predictions are. Both of these are in the same units as the thing you're predicting (e.g. dollars), so they're easy to interpret.

MAE — Mean Absolute Error

The average size of the miss. If MAE is 46, your predictions are off by about 46 units on average. Every miss counts equally.

RMSE — Root Mean Squared Error

Similar, but it squares the errors before averaging, which makes **big misses hurt much more** than small ones. If occasional large errors are dangerous, RMSE is the stricter judge. RMSE is always greater than or equal to MAE; a big gap between them warns you of a few large, costly mistakes.

Here are both, measured for real on a model predicting a health score:

REAL OUTPUT — REGRESSION ERROR

```
MAE : 46.2      (typical miss is about 46 units)
RMSE: 58.5      (higher than MAE -> some larger misses are present)
example: true value = 321.0   model predicted = 238.5
```

Metric	Predicts...	Best value	Use when
Accuracy	Category	1.0 (100%)	Classes are balanced
Precision	Category	1.0	False alarms are costly
Recall	Category	1.0	Misses are costly
F1	Category	1.0	You need both at once
AUC-ROC	Category (ranking)	1.0	Judging across all cut-offs
MAE	Number	0	All misses count equally

RMSE	Number	0	Big misses must be punished
------	--------	---	-----------------------------

The Flower Sorter

Train your first real model, test it honestly, and watch overfitting happen with your own eyes.

This project ties together every idea in Phase 1: it's **supervised classification**, you'll see **training vs inference**, and you'll judge it with a **confusion matrix**, **accuracy**, **precision and recall** — then push it into **overfitting** on purpose. The data is built into scikit-learn, so there's nothing to download.

The task

The famous *Iris* dataset contains 150 flowers from three species, each described by four measurements (petal and sepal length and width). We'll teach a decision tree to look at the four measurements and name the species.

0 Set up (one-time)

Install Python from python.org if you don't have it, then in a terminal run:

```
pip install scikit-learn
```

Save the code below as `flower_sorter.py` and run it with `python flower_sorter.py`.

1 The full program

```
# The Flower Sorter – your first machine learning model
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1) LOAD the data (built in – no download)
data = load_iris()
X = data.data          # the 4 measurements for each flower
y = data.target        # the correct species (0, 1 or 2)

# 2) SPLIT – hide 20% of flowers so we can test honestly
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3) TRAIN – the model studies only the training flowers
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# 4) PREDICT (inference) on flowers it has NEVER seen
predictions = model.predict(X_test)

# 5) EVALUATE
print("Accuracy:", accuracy_score(y_test, predictions))
```

```
print(confusion_matrix(y_test, predictions))
print(classification_report(
    y_test, predictions, target_names=data.target_names))
```

2 What each step is really doing

Line	Plain-English meaning
<code>load_iris()</code>	Loads the 150 example flowers and their correct answers.
<code>train_test_split</code>	The honesty step. Sets aside 20% of flowers the model won't be allowed to study, so we can test it on genuinely unseen data — exactly how we catch overfitting.
<code>model.fit(...)</code>	Training. The tree learns its yes/no questions from the training flowers only.
<code>model.predict(...)</code>	Inference. The trained tree names the species of the hidden test flowers.
<code>...report</code>	Compares predictions to the true answers and prints the metrics from Part C.

3 What you'll see

REAL OUTPUT

Accuracy: 1.0

Confusion matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30

Every number on the diagonal of the confusion matrix is a correct call; every `0` off the diagonal means zero mistakes. The model named all 30 unseen flowers correctly. (Iris is an easy, clean dataset — don't expect a perfect `1.00` on messy real-world data!)

4 Make overfitting happen on purpose

Change one thing — the tree's `max_depth` — and print the score on the training data *and* the test data. This reproduces the A.4 demo yourself:

```
for depth in [1, 3, 20]:
    m = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train, y_train)
    train = accuracy_score(y_train, m.predict(X_train))
```

```
test = accuracy_score(y_test, m.predict(X_test))
print(depth, "->", round(train,2), round(test,2))
```

At depth 1 both scores are low (**underfitting**). At a sensible depth they're high and close. At depth 20 the training score hits a perfect 1.0 while the test score doesn't improve — the model is **memorising**, not learning. You've now seen the most important lesson in ML with your own data.

STRETCH CHALLENGE

1. Swap `DecisionTreeClassifier` for `LogisticRegression` (from `sklearn.linear_model`) and compare. 2. Change `test_size` to `0.4` — does the score change? 3. Print `model.predict([[5.1, 3.5, 1.4, 0.2]])` to classify a single made-up flower.

PHASE 1 CHECKLIST — YOU CAN NOW...

✓ explain AI vs ML vs deep learning · ✓ name the three learning styles · ✓ separate training from inference · ✓ spot overfitting · ✓ pick an algorithm by its job · ✓ read accuracy, precision, recall, F1, AUC, RMSE & MAE · ✓ train, test and evaluate a real model.

Plain-English glossary

Term	In plain words
Model	The thing that learns from data and then makes predictions.
Feature	One input measurement (e.g. petal length). Features are the clues the model uses.
Label / target	The correct answer you're trying to predict.
Training	Teaching the model by showing it labelled examples.
Inference / prediction	Using the trained model to answer a new question.
Classification	Predicting a category (spam / not spam).
Regression	Predicting a number (a price).
Generalisation	Doing well on data the model never saw — the real goal.
Overfitting	Memorising the training data instead of learning the pattern.
Underfitting	Being too simple to capture the pattern at all.
Train/test split	Hiding some data so you can test the model honestly.
scikit-learn	The standard, free Python library for classic machine learning.

What's next — Phase 2

Python for ML: Practitioner Level. Now that you have the mental model, Phase 2 gives you the hands. You'll learn to wrangle real data with **pandas** and **NumPy**, handle missing values and outliers, do basic feature engineering, visualise data with **Matplotlib** and **Seaborn**, and run the full scikit-learn workflow — train/test split, pipelines and cross-validation — on a brand-new hands-on project.

Machine Learning, From Zero · Phase 1 of 4 · Definitions follow the official scikit-learn 1.8 documentation (scikit-learn.org). This guide is for learning and does not reproduce copyrighted text from any source.